**Deduction for Late Submission:****Final Mark:**%

SMM636 - Group Assignment 01 (Group 01)

Contents

1 Purpose & context	3
2 Methods	3
2.1 Dataset and prediction task	3
2.2 Modelling approach	4
3 Results & discussion	5
4 Conclusion	8
 Appendices	 8
A GitHub Resources	9
B Overall of R Shiny application	10
C Generative AI Declaration	11
C.1 Tools Used	11
C.2 Prompts used	11
C.2.1 Use of GenAI outputs & alignment with guidelines	11
C.3 Changes made and statement of originality	12

1 Purpose & context

Machine learning algorithms are frequently perceived as ‘black boxes’ by stakeholders who need to understand why a model makes a particular decision. Tree-based methods provide a practical bridge as a single decision tree can be read intuitively like a flowchart, whilst ensemble variants (Random Forest and XGBoost) typically improve predictive accuracy but reduce direct interpretability.

To provide a clear framework for interpretation, this project adopts a consulting scenario. We act as a team of actuaries working for (ABBA) Actuarial Consulting¹, pitching our consulting data analytics capabilities to prospective clients using a previous case study. An interactive R Shiny app was developed to guide clients through (i) how DTs work, (ii) how ensemble methods work, and (iii) how to discuss the trade-offs in an accessible, client facing way.

The case study uses the publicly available Cleveland Heart Disease dataset as a real world example of a binary classification problem (ritwik_b3, 2023). This dataset is used solely as an illustrative case study to demonstrate methodology, but the approach described here is directly transferable to actuarial applications, such as claims triage and fraud detection.

2 Methods

2.1 Dataset and prediction task

The Cleveland Heart Disease data used for this project contains 303 patient records and 14 variables (13 predictors and 1 outcome). As shown in Table 1, the predictors include a mixture of clinical information that mirrors the heterogeneity of many real-world business datasets.

Predictors were treated as either quantitative or categorical to support modelling across different tree-based algorithms. Decision-tree and random forest implementations natively handle categorical predictors, but for XGBoost, categorical predictors were encoded using dummy variables.

In this case study, the binary classification task is to predict whether a patient has heart disease, regardless of severity. The class balance is 54.1 % healthy and 45.9% disease, which is sufficiently mixed to meet the coursework criteria and illustrate how tree-based methods work, including the trade-offs between sensitivity, specificity and interpretability.

¹Abudlmatin, Ben, Basmah, and Ardi Actuarial Consulting (trademark pending)

Table 1: Predictors in the Cleveland Heart Disease dataset, grouped by clinical theme (demographics and symptoms, clinical vitals, and test results).

Variable	Type	Description (group)
age	Numeric (continuous)	Patient age in years
sex	Categorical (binary)	Sex (0 = female, 1 = male)
cp	Categorical (nominal)	Chest pain type (0–3)
trestbps	Numeric (continuous)	Resting blood pressure (mm/Hg)
chol	Numeric (continuous)	Serum cholesterol (mg/dl)
fbs	Categorical (binary)	Fasting blood sugar > 120 mg/dl
restecg	Categorical (nominal)	Resting ECG result
thalach	Numeric (continuous)	Maximum heart rate achieved
exang	Categorical (binary)	Exercise-induced angina
oldpeak	Numeric (continuous)	ST depression induced by exercise relative to rest
slope	Categorical (ordinal)	Slope of peak exercise ST segment
ca	Categorical (ordinal)	Number of major vessels coloured by fluoroscopy
thal	Categorical (nominal)	Thalassemia status

2.2 Modelling approach

A 70/30 train-test split was employed, taking care to preserve the prevalence of heart disease across partitions. Critically, all models were trained using only the testing set, with final performance metrics reported using the unseen test data to ensure generalisability.

Three tree-based approaches/architectures were compared:

1. Decision Tree: the simplest model, where a single interpretable tree is grown and then pruned to reduce overfitting, reflecting the standard bias-variance trade-off
2. Random Forest: an ensemble approach fitting x trees using bootstrapping samples. Predictions are made by majority vote.
3. XGBoost (boosted trees): builds shallow trees sequentially, with each tree focusing on the errors the previous trees still make.

All models were implemented in R. The decision tree recursively partitioned the feature space into homogeneous groups by maximising impurity reduction. To mitigate overfitting, the tree was pruned using rpart's internal 10-fold cross-validation.

For the ensemble methods (Random Forest and XGBoost), hyperparameters were determined offline using standard cross-validation techniques. However, for the purposes of the client-facing web application, these parameters are controlled dynamically via interactive sliders. This was an intentional design choice to demonstrate model tuning and the value of ensemble averaging, allowing clients to

explore the impact of these settings in real time (Breiman, 2001).

3 Results & discussion

The decision tree classifier is shown in Figure 1. On the unseen testing data, this achieved a sensitivity of 76%, alongside a specificity of 82% and a precision of 78%.

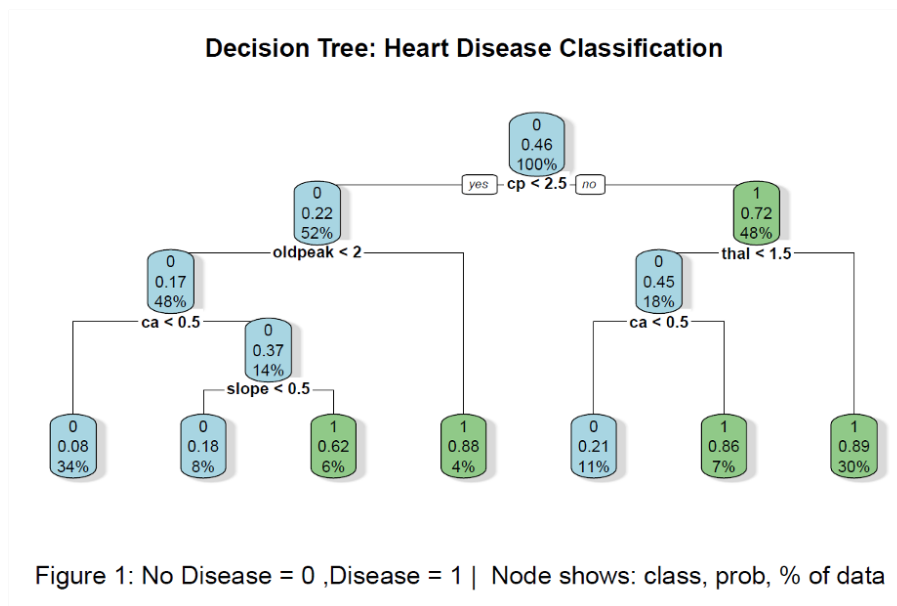


Figure 1: Pruned decision tree predicting heart disease, displaying (top to bottom) node classifications, class probabilities, and the percentage of the dataset within each node.

Figure 2 presents the cross-validation relative error across different cp values and their corresponding tree sizes. The error decreases steadily as tree complexity increases, reaching its minimum at $cp = 0.012$ with a tree size of 7 terminal nodes. The dotted horizontal line represents the minimum cross-validation error plus one standard error (1-SE rule), which is first crossed at $cp = 0.031$ corresponding to a tree size of 5 nodes. Applying the minimum cross-validation error criterion, the optimal cp was identified as 0.012, which closely aligns with $rpart$'s default cp threshold of 0.01. Consequently, the pruned tree retained all 7 nodes, confirming that the default tree was already the cross-validation-optimal solution and no further pruning was necessary.

The predictive performance of each modelling approach on the unseen testing data is shown in Table 2. While the Decision Tree supports direct tracking of a single patient's logical path, the Random Forest and XGBoost ensembles yield stronger discriminative performance, as reflected by their ROC curves and corresponding AUCs (Figure 3).

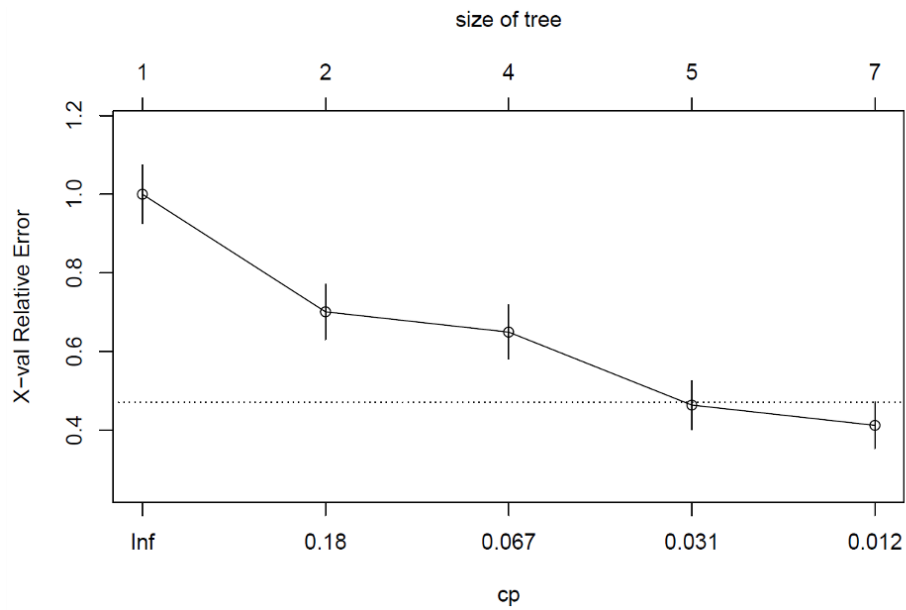


Figure 2: Cross validation error by complexity parameter (cp).

Table 2: Performance metrics evaluated on the 30% testing partition.

ML method	Accuracy	Sensitivity	Specificity
Decision Tree	0.793	0.762	0.820
Random Forest	0.826	0.775	0.878
XGBoost	0.815	0.738	0.880

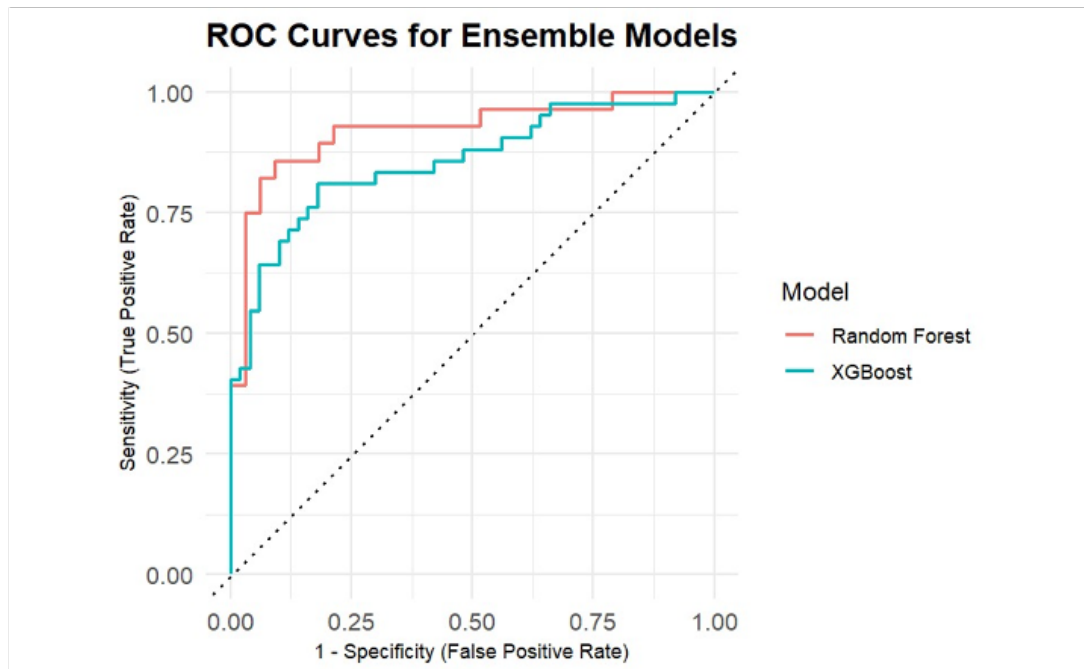


Figure 3: Receiver Operating Characteristic (ROC) curves evaluated on the unseen testing data, illustrating the superior discriminative performance of the Random Forest and XGBoost ensemble models.

Demystifying the black box: SHAP values

However, improved ensemble accuracy comes at the cost of direct transparency. Unlike a single decision tree, individual XGBoost trees do not provide a single, end-to-end flowchart decision rule; their leaf nodes contribute raw log-odds scores that are aggregated across trees to produce a final prediction.

The predicted probability of belonging to the disease class is derived via the logistic function

$$p(X) = \frac{1}{1 + \exp(-Y)}$$

where Y is the total aggregated score (log-odds). Because XGBoost optimises an objective using first- and second-order derivatives (gradient and Hessian) of this loss, a simple, single flowchart explanation for the full boosted ensemble is not feasible in the same way as a pruned decision tree. To bridge this interpretability gap for clients, SHAP (SHapley Additive exPlanations) values were used. SHAP decomposes the total score Y into additive contributions from individual features, providing a clear, quantitative explanation of how each predictor influences the final prediction.

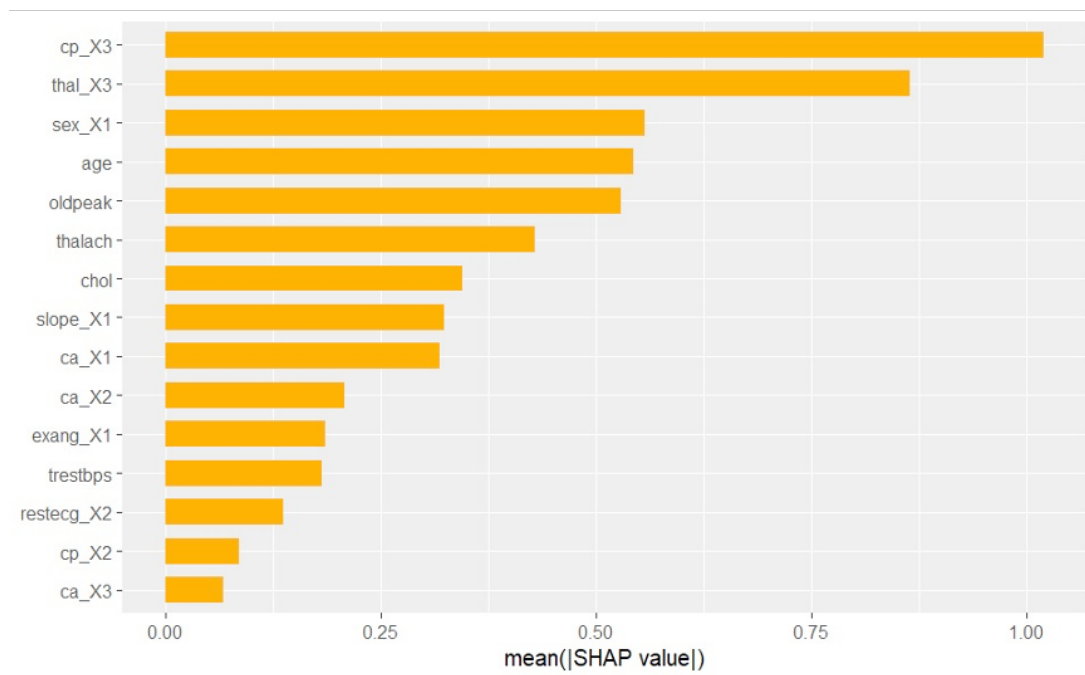


Figure 4: SHAP summary bar chart ranking the overall influence of clinical features on the XGBoost model's predictions.

To provide deeper context beyond global importance, a SHAP beeswarm plot was also generated. Each point in Figure 5 represents an individual patient, illustrating both the magnitude and directional impact of specific feature values on the prediction. This supports client discussion not only of *which*

features matter (e.g., chest pain type), but *how* particular values push predictions toward or away from a heart disease outcome.

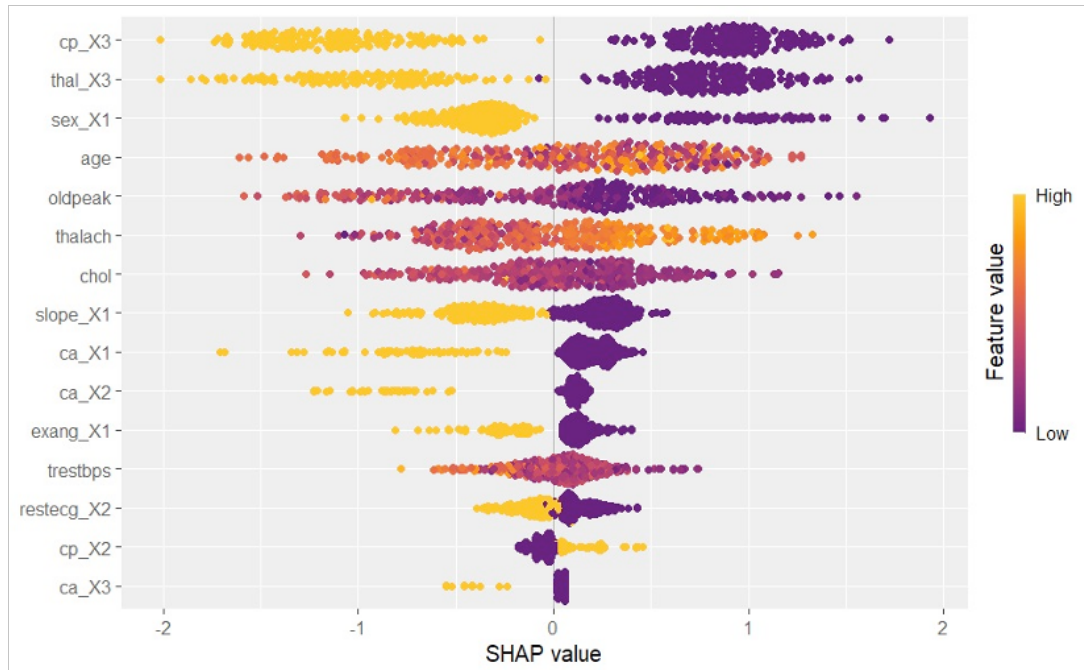


Figure 5: SHAP beeswarm plot. Each point represents an individual patient, showing the magnitude and direction of each feature's contribution to the model output.

4 Conclusion

By contrasting the transparent logic of pruned Decision Trees with the stronger predictive performance of ensemble methods, this project illustrates how actuaries can communicate key machine-learning trade-offs to non-technical stakeholders. When interpretability and auditability are essential, simpler models may be preferred and when predictive performance is the priority, ensembles may be more suitable, provided appropriate explanation tools (such as SHAP) are used.

References

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. Available at: <https://link.springer.com/article/10.1023/A:1010933404324>. (Accessed: February 27, 2026).
- ritwik_b3. (2023). *Heart disease cleveland* [Kaggle dataset.]. Available at: <https://www.kaggle.com/datasets/ritwikb3/heart-disease-cleveland>. (Accessed: February 23, 2026).

A GitHub Resources

All resources described in these appendices are available via a public Github repository:

<https://github.com/ytterbiu/SMM636-ML-group-coursework-01-g01>

Please note that this commit history reflects updates made to the project repository and is not a comprehensive timeline of all contributions by group members.

The R Shiny Apps created for this project are available via:

- <https://3enji.shinyapps.io/smm636-a01-tree-based-methods/>
- <https://3enji.shinyapps.io/smm636-a01-tree-based-methods-standalone-tab4/>

The source code for these apps are available via the project repository:

- [https://github.com/ytterbiu/SMM636-ML-group-coursework-01-g01/tree/main/
release-branch](https://github.com/ytterbiu/SMM636-ML-group-coursework-01-g01/tree/main/release-branch)
- [https://github.com/ytterbiu/SMM636-ML-group-coursework-01-g01/tree/main/
release-standalone-tab4](https://github.com/ytterbiu/SMM636-ML-group-coursework-01-g01/tree/main/release-standalone-tab4)

B Overall of R Shiny application

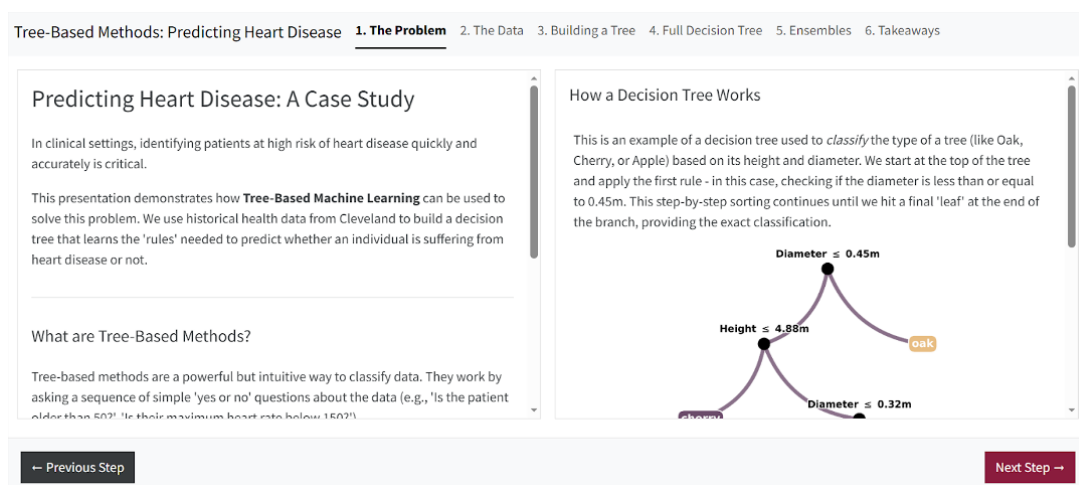


Figure 6: The primary R Shiny application interface.

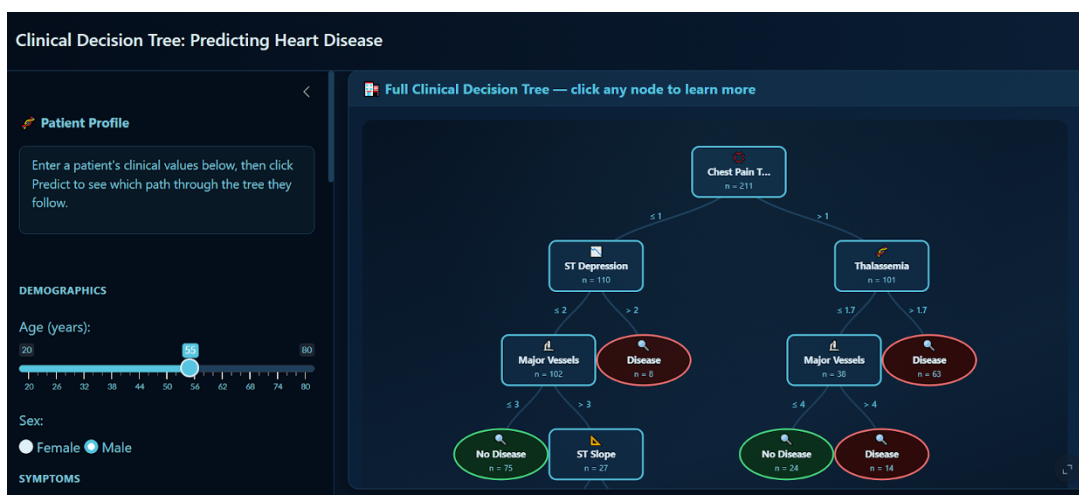


Figure 7: The primary R Shiny application interface.

Two interactive Shiny applications were developed from this analysis. The first (shown in Figure 6) is an educational application designed for teaching and presentations, providing a guided, multi-tab walkthrough of the full modelling workflow, starting by addressing the problem definition and data exploring ending with decision tree construction, ensemble methods, and key insights. It is intended for classroom or conference audiences who may be unfamiliar with machine learning, and includes a link to the second application.

The second is a standalone clinical decision tool aimed at medical practitioners (Figure 7). It allows users to input a patient's clinical profile and obtain a real-time heart disease risk prediction based on the trained decision tree. The application also visualizes the tree interactively, enabling clinicians to trace the exact decision path for each patient, and is designed for independent deployment.

C Generative AI Declaration

Generative AI (GenAI) tools were used throughout this project strictly to assist with conceptual understanding of machine learning algorithms, R programming syntax, and debugging complex software interactions. In practice, these tools functioned as a technical sounding board, helping to troubleshoot code and explain statistical mechanics.

C.1 Tools Used

Google Gemini (Gemini 3.1 Pro)

C.2 Prompts used

The GenAI tools were provided with extracts of our R code, Shiny UI layouts, and error messages generated by the R console. A non-exhaustive list of prompts used is shown below, highlighting the *types* of technical queries submitted rather than an exhaustive transcript:

- “Why is my rpart predict function throwing a 'dimnames applied to non-array' error when I try to predict a single patient row?”
- “How can I force a Shiny UI to redraw an HTML widget? The DiagrammeR tree plot isn't updating when I move the slider.”
- “Can you help me fix the 'expanded path length 1024' warning when plotting an XGBoost tree?”
- “What is the standard way to plot an ROC curve with the x-axis anchored strictly from 0 to 1 using ggplot2?”

C.2.1 Use of GenAI outputs & alignment with guidelines

The GenAI tools served as a “consultant” member of the group. Notably, the AI output:

- Helped debug obscure package conflicts (e.g., between tidymodels, xgboost, and Shiny widget caching).
- Suggested code workarounds for known R package limitations (such as duplicating single-row inputs to bypass rpart matrix dimension bugs).

- Aided our understanding of how to implement SHAP values to explain complex ensemble models.
- Assisted in designing a clean, user-friendly UI layout using the bslib package in Shiny.

C.3 Changes made and statement of originality

All AI suggestions were critically reviewed and debated by the group before implementation. For example:

- All generated R code was manually tested, reviewed, and adapted to fit our specific variable names and dataset structure.
- Where the AI explained concepts (such as the trade-off between interpretability and accuracy), these explanations were used to build our own understanding. The final prose, business framing, and arguments presented in the report were written entirely by the human authors in our own words.

While GenAI was treated as a collaborator, the final analysis, statistical interpretation, and the written content of this report were generated and verified by the (human) group members.